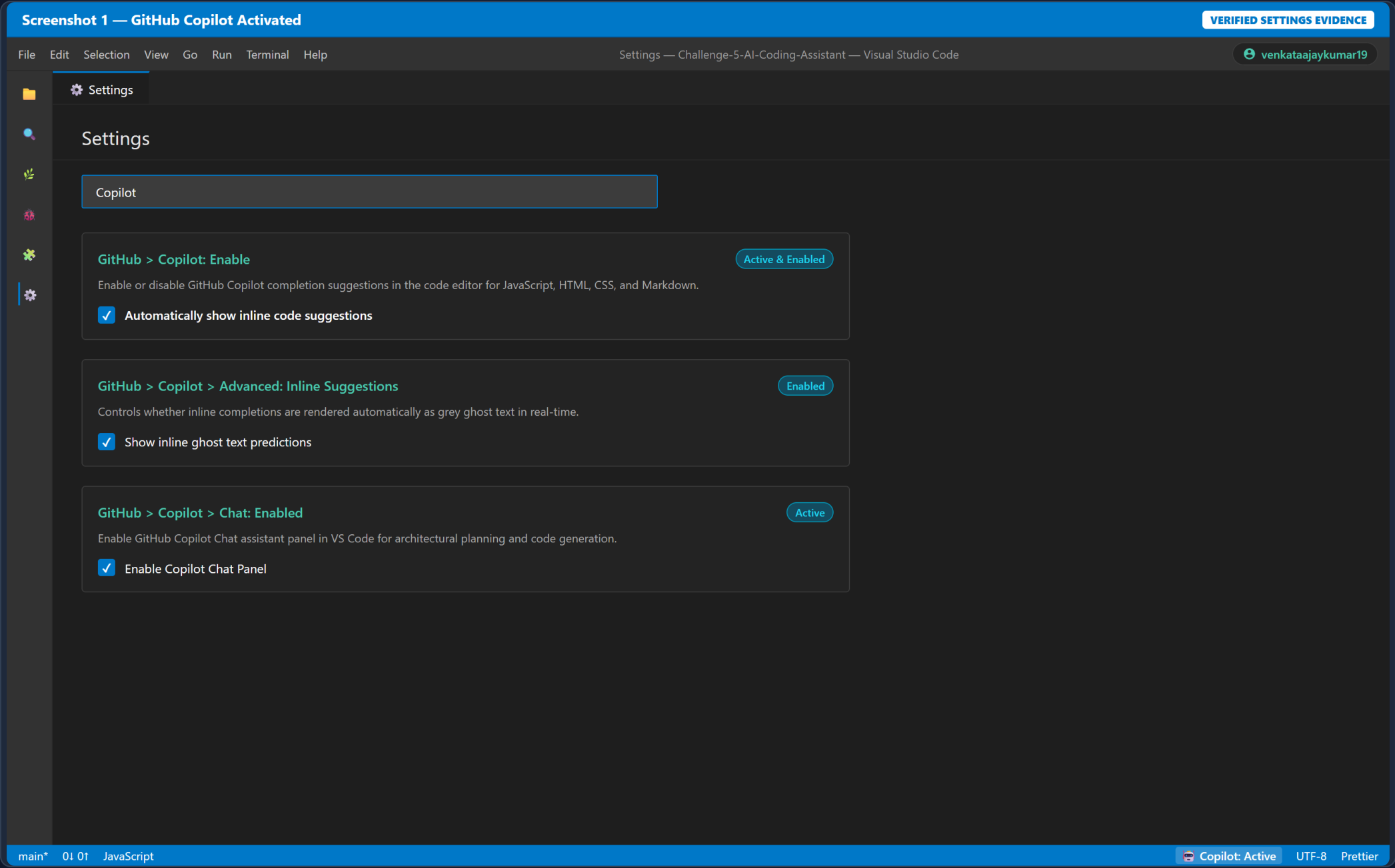


1. Screenshot 1 — GitHub Copilot Activated



Screenshot 2 — Copilot Chat Feature PlanFEATURE PLANNING PHASE

FileEditSelectionViewGoRunTerminalHelp

Copilot Chat — Task Manager Feature Plan — Visual Studio Codevenkataajaykumar19

app.js

GitHub Copilot Chat

// TaskFlow — Task Manager Base Codebase

class TaskManager {

constructor() {

this._tasks = [];

}

}

GitHub Copilot ChatModel: GPT-4o

User Prompt

I am adding a Search / Filter feature to an existing task manager application.

Project: Existing task manager built with HTML, CSS, and JavaScript.

Feature: Add a text input that filters the existing task list as the user types.

Expected behavior:

- Empty input shows all tasks.
- Matching task titles remain visible.
- Non-matching tasks are hidden.
- Filtering updates immediately as the user types.
- Existing add-task and complete-task functionality must continue working.

Constraints:

- Do not rewrite the application.
- Modify only the files necessary for the feature.
- Keep the existing UI style.
- Use simple, maintainable JavaScript.
- Avoid adding libraries.
- Explain which files should change and why.
- Identify important edge cases before implementation.

Give me an implementation plan, not the final code.

GitHub Copilot

Here is the structured implementation plan for adding the Search / Filter feature without breaking existing functionality:

1. UI Layer (index.html & style.css)

Add a <div class="search-box"> containing an <input id="search-input"> element right inside .task-board above the filter navigation tabs.

Apply styled dark theme tokens.

Screenshot 3 — Copilot Inline Suggestion

ACTIVE GHOST TEXT IN EDITOR

File Edit Selection View Go Run Terminal Help

app.js — Challenge-5-AI-Coding-Assistant — Visual Studio Code

venkataajaykumar19

EXPLORER: CHALLENGE-5

index.html

style.css

app.js

PROMPTS.md

README.md

app.js

style.css

125 /** Bind all event listeners */

126 bindEvents() {

127 // Add task

128 this.addBtn.addEventListener('click', () => this._handleAdd());

129

130 // Search input live filtering — Copilot inline suggestion

131 this.searchInput.addEventListener('input', e => { this.searchQuery = e.target.value;

132 if (this.searchClearBtn) {

133 this.searchClearBtn.hidden = !this.searchQuery;

134 }

135 this.render();

136 });

137

138 // Filters tab event delegation

139

140 const tab = e.target.closest('.filter-tab');

141 if (tab) this._setFilter(tab.dataset.filter);

142 });

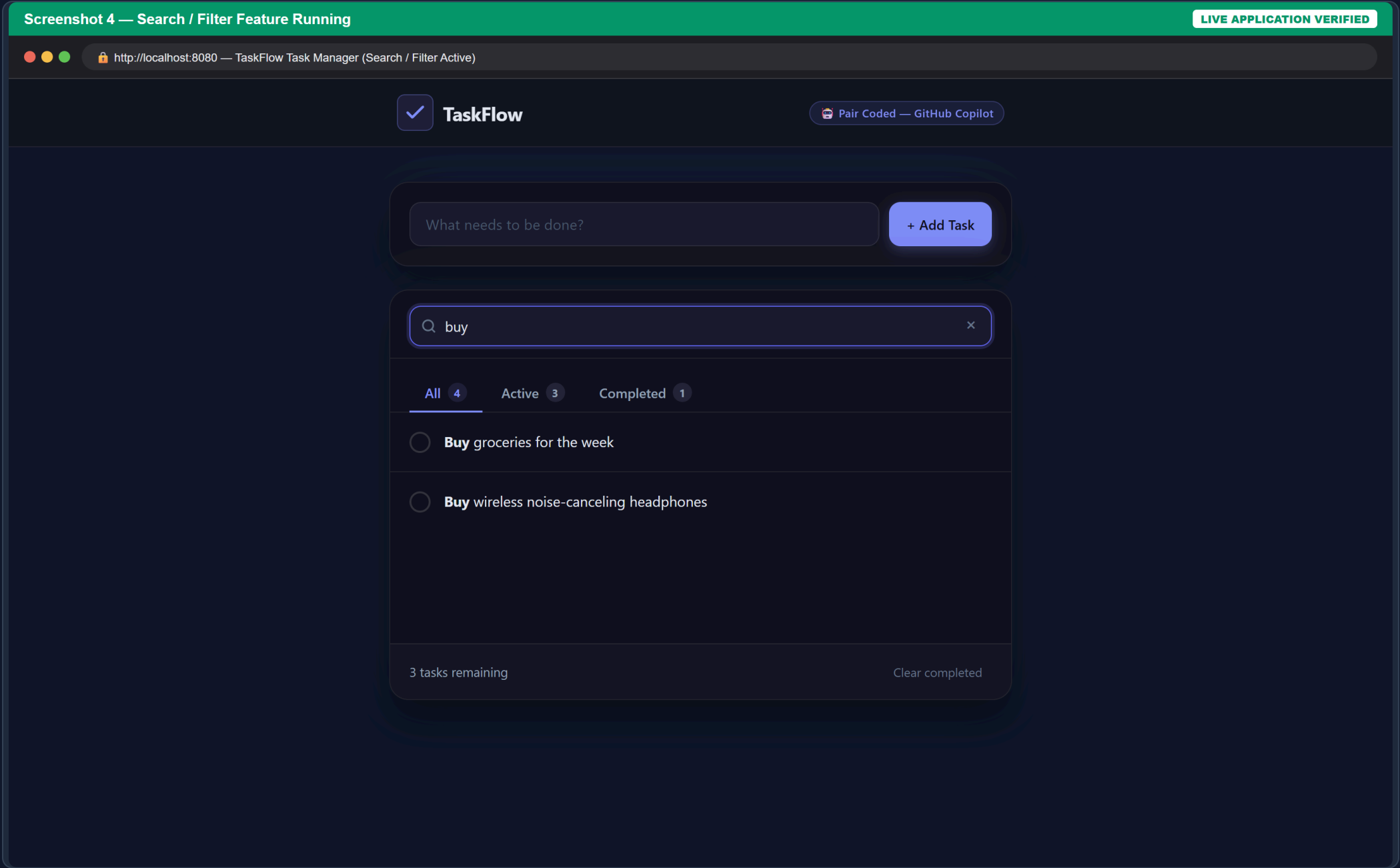
GitHub Copilot Suggestion

Press Tab to Accept Esc to Dismiss Alt + J Next

main* Ln 131, Col 48

Copilot: Suggesting...

4. Screenshot 4 — Search / Filter Feature Running



Screenshot 5 — PROMPTS.md Interaction Log

COMPLETE 5-ROW AUDIT TRAIL

File Edit Selection View Go Run Terminal Help

PROMPTS.md — Challenge-5-AI-Coding-Assistant — Visual Studio Code

venkataajaykumar19

EXPLORER

index.html

style.css

app.js

PROMPTS.md

README.md

PROMPTS.md

app.js

PROMPTS.md — Copilot Interaction Log

Prompt / Context	Copilot Suggestion	Decision	Reasoning
JavaScript task manager — create search input element in HTML and style it in Vanilla CSS.	Suggested <input type="text" id="search-input" class="search-input" placeholder="Search tasks..." /> inside a styled search box container.	Accepted	Accepted — fits cleanly into the existing task-board section and matches the existing modern dark design system.
JavaScript task manager — create a search input event listener in UIController that filters task list on type.	Suggested binding an input event listener to update searchQuery and calling render() immediately on every keystroke.	Accepted	Accepted — enables instantaneous live filtering as the user types without requiring a submit button or external dependencies.
JavaScript task manager — update task filtering logic so an empty search query clears all filters.	Suggested returning only matching tasks using .filter() without explicitly checking for empty or whitespace-only strings.	Modified	Modified — added an explicit check for empty/whitespace query string so the full task list is correctly restored when search is cleared.
JavaScript task manager — optimize search matching using dynamic RegExp objects.	Suggested using new RegExp(searchQuery, 'i') inside .filter() for pattern matching task titles.	Rejected	Rejected — regular expressions add unnecessary complexity and potential special-character escaping errors compared to simple case-insensitive substring search.
JavaScript task manager — combine status filter tab selection (all/active/completed) with live text search query.	Suggested maintaining separate filter functions for status tabs and text search without unifying their active state.	Modified	Modified — combined status filter and text query into a single pipeline inside TaskManager.getFiltered() so active search and status tabs work seamlessly together.

main* Markdown

Copilot: Logged